Student notes — Session 4 · else-if, switch and worked examples

Slides: [Week_02_Programming_Principles_I_C.pdf](../slides/Week_02_Programming_Principles_I_C.pdf). These notes are the notes pane of the student copy of the deck.

Slide 13 — else-if chains

The quiz papers come back on Tuesday, marked.

How to try the examples in these notes. Same as last week. Open Visual Studio Code, open your folder, make a new file ending in `.c`, then in the terminal type `gcc -Wall name.c -o name` and press Enter, then `./name` (or `.\name` in Windows PowerShell) and press Enter.

Two paths were not enough. On Tuesday you saw `if` and `if/else`: one condition, two paths. Grades have four categories. For that C has the **chain**: `if`, then `else if`, then `else if`, then `else`.

Example 1. The program on the slide.

```
#include <stdio.h>
int main(void) {
    int score;
    printf("Score: ");
    scanf("%d", &score);
    if (score >= 90) {
        printf("A\n");
    } else if (score >= 80) {
        printf("B\n");
    } else if (score >= 70) {
        printf("C\n");
    } else {
        printf("Needs improvement\n");
    }
    return 0;
}
```

Run it several times with different scores:

You type	Screen shows	Why
`95`	`A`	`95 >= 90` is true. Print `A`. The chain stops.
`90`	`A`	`90 >= 90` is true.
`85`	`B`	`85 >= 90` is false, so go on. `85 >= 80` is true. Print `B`. Stop.
`79`	`C`	not 90, not 80, but `79 >= 70` is true.
`70`	`C`	`70 >= 70` is true.
`69`	`Needs improvement`	none of the three conditions is true, so the `else` runs.

The two rules.

1. C checks the conditions **from the top**.
2. **The first true condition wins.** Its block runs, and then the whole chain is finished. Nothing below it is looked at.

So for `95`, only `A` prints. It is true that `95 >= 80` too, but C never gets there. It stopped after `A`.

A chain is not the same as three separate `if`s. Look at this:

```
int score = 95;
if (score >= 70) {
    printf("C\n");
}
if (score >= 80) {
    printf("B\n");
}
if (score >= 90) {
    printf("A\n");
}
```

The screen shows:

```
C
B
A
```

Three separate `if`s, no `else`. Each one is checked on its own, and all three are true for 95. With a chain (`else if`), exactly **one** path runs. Never two. And if the chain ends with `else`, never none.

Try it yourself. Write a program that reads a temperature (a whole number) and prints `Hot` above 30, `Warm` above 20, otherwise `Cold`. Test with `35`, `25`, `10`. (Expect `Hot`, `Warm`, `Cold`. For `25`: `25 > 30` is false, `25 > 20` is true, so `Warm`.)

Slide 14 — Order can break your logic

A program can compile and still be wrong. The compiler checks that your C is spelled correctly. It does not check that your thinking is correct.

Example: the same chain in the wrong order.

```
if (score >= 70) {
    printf("C\n");
} else if (score >= 80) {
    printf("B\n");
} else if (score >= 90) {
    printf("A\n");
}
```

This compiles with no warnings. Now trace it for `score = 95`:

1. Is `95 >= 70`? Yes. Print `C`. The chain stops.

So a score of 95 prints `C`. The `80` and the `90` are never reached. Not for 95. Not for any score. Every score of 70 or more prints `C`. `A` and `B` can **never** print.

You type	Wrong-order chain prints	Correct chain prints
95	C	A
85	C	B
75	C	C
50	nothing	Needs improvement

(The wrong-order chain has no `else`, so 50 prints nothing at all.)

The fix. Put the **strongest** condition first. For "at least" conditions (`>=`), that means the biggest number first: 90, then 80, then 70. For "less than" conditions (`<`), it means the smallest number first. You see that in the BMI program on slide 18.

Rule to remember: the condition that catches the **fewest** cases goes first. If a wide condition comes first, it catches everything and the narrow ones never run.

Try it yourself. Take the wrong-order chain above, add `printf("Done\n");` after it, and run it with 95. You see `C` and `Done`. Now swap the conditions into the right order and run again. You see `A` and `Done`.

Slide 15 — Nested decisions

Sometimes the second question only makes sense after the first one. "Are you logged in?" And only if yes: "are you the admin?" If you are not logged in, asking about admin makes no sense.

So you put an `if` **inside** an `if`. That is called **nesting**.

Example: the program on the slide.

```
#include <stdio.h>
int main(void) {
    int loggedIn = 1;
    int isAdmin = 0;
    if (loggedIn) {
        if (isAdmin) {
            printf("Admin panel\n");
        } else {
            printf("User page\n");
        }
    } else {
        printf("Please log in\n");
    }
    return 0;
}
```

The screen shows:

```
User page
```

Trace it: `loggedIn` is 1, which counts as true, so C goes inside the outer block. There, `isAdmin` is 0, which counts as false, so the inner `else` runs: `User page`.

All four cases.

loggedIn	isAdmin	Screen shows
1	1	Admin panel
1	0	User page
0	1	Please log in
0	0	Please log in

Look at the last two rows. When `loggedIn` is 0, the inner `if` is never even looked at. It does not matter what `isAdmin` is.

Why `if (loggedIn)` and not `if (loggedIn == 1)`? On Tuesday: 0 is false, anything else is true. `loggedIn` is 1, so `if (loggedIn)` means "if loggedIn is not zero". Both ways work. The short way is common in C.

Count your braces. Every `{` needs its `}`. Indent each level one step further in, as the slide does. The compiler does not care about indentation. You do, when you read it next week.

Do not nest too deep. Two levels are fine. Four levels are hard to read. Later you learn ways to avoid it.

Try it yourself. Change the two values to `loggedIn = 0` and `isAdmin = 1`. Before you run it, say what prints. (Answer: `Please log in.`)

Slide 16 — switch statements

A menu: press A to add an item, press B to view the cart. One variable, compared against a few **fixed** values. You could do it with `else if`. But C has a shape made for exactly this: `switch`.

Example: the program on the slide.

```
#include <stdio.h>
int main(void) {
    char option;
    printf("A/B/C: ");
    scanf(" %c", &option);
    switch (option) {
        case 'A':
            printf("Add item\n");
            break;
        case 'B':
            printf("View cart\n");
            break;
        default:
            printf("Unknown option\n");
    }
    return 0;
}
```

Run it three times:

You type	Screen shows
`A`	`Add item`
`B`	`View cart`
`D`	`Unknown option`

Also: `a` (small letter) shows `Unknown option`. `'a'` and `'A'` are different characters.

How to read a `switch`.

- `switch (option)`: look at the box `option`.
- `case 'A':` if the box holds `'A'`, start running from here.
- `break;` stop, leave the `switch`.
- `default:` if no case matched, run this. Always write a `default`, or an unknown letter prints nothing at all.

What `break` does. Try this. Delete both `break;` lines, compile, and run with `A`. The screen shows:

```
Add item
View cart
Unknown option
```

All three lines. Without `break`, C starts at `case 'A'` and **keeps going down** through every case below it until the end of the `switch`. This is called **falling through**. It is almost never what you want. Put the `break` lines back. (Run it with `B` and no `breaks`: `View cart` and `Unknown option`.)

The space in `" %c"`. Look at `scanf(" %c", &option);`. There is a space before `%c`, inside the quotes. It is there on purpose.

When you type `A` and press Enter, two things go in: the letter `A` and the Enter key. `%d` and `%lf` skip spaces and Enters by themselves. `%c` does **not**. It takes exactly the next character, whatever it is. So if an earlier `scanf` left an Enter behind, `%c` reads that Enter, and your box holds a new line instead of a letter. The space before `%c` says "skip spaces and Enters first". **Rule: when reading one character, always write `" %c"` with the space.**

Here is what goes wrong without it:

```
int n;
char option;
printf("Number: ");
scanf("%d", &n);
printf("A/B/C: ");
scanf("%c", &option);
printf("You chose [%c]\n", option);
```

You type `5`, Enter, then `B`, Enter. The screen shows `You chose [` and then a new line and `]`. The box got the Enter, not the `B`. With `" %c"` it shows `You chose [B]`.

`switch` versus `else if`. `switch` only compares against fixed values: `'A'`, `'B'`, `1`, `2`. It cannot do `case score >= 90`. For ranges, use `else if`. For a fixed list of choices, use `switch`.

Try it yourself. Add a `case 'C':` that prints `Checkout` (with its `break`). Run with `C`. Then run with `c` and see `Unknown option`.

Slide 17 — Common Week 2 bugs

Six bugs. You will make every one of them. Here is each one with what it looks like and how to see it.

1. `=` instead of `==`.

```
int x = 3;
if (x = 5) {
    printf("five\n");
}
printf("x is %d\n", x);
```

The screen shows:

```
five
x is 5
```

`x = 5` is not a question. It is an order: put 5 into the box. Then C looks at the value, 5. Five is not zero, so it counts as true. The block runs, **and `x` is now 5**, not 3. The condition changed your variable.

With `-Wall`, the compiler warns you: `warning: suggest parentheses around assignment used as truth value`. It is a warning, not an error, so the program still builds. When you see that warning, you almost certainly meant `==`.

Rule: inside a condition, always two equals signs.

2. Missing braces. Without `{ }`, only **one** line belongs to the `if`.

```
if (x >= 0)
    printf("positive");
    printf("done");
```

The second `printf` is indented as if it belonged to the `if`. It does not. It runs every time, whatever `x` is. With `-Wall` the compiler warns: `this 'if' clause does not guard...`. **Rule: always write the braces, even for one line.**

3. Wrong format specifier. `scanf("%d", &weight)` for a `double` box. Last week's bug. Use `%lf` for `double` in `scanf`.

4. Integer division inside a `double`.

```
int sum = 7, count = 2;
double avg = sum / count;
printf("%.2f\n", avg);
```

Prints `3.00`, not `3.50`. `sum / count` is `int / int`, so it is 3 before it is stored. Fix: `(double) sum / count` prints `3.50`. From Tuesday.

5. Wrong `else if` order. Slide 14. Lower threshold first, and the higher ones never run.

6. Forgetting `break`. Slide 16. The `switch` falls through into the next case.

The debugging habit. When the output is wrong, do not stare at the whole program. Pick **one** input, for example `95`, and trace it with a pen, line by line, writing down what each box holds. You will find the bug.

Slide 18 — Worked example: BMI category

A whole program: read two numbers, calculate, then choose a path.

Step 1: the plan, before the code.

1. Read weight (kg) and height (m).
2. BMI = weight ÷ (height × height).
3. Under 18.5: Underweight. Under 25: Normal. Under 30: Overweight. Otherwise: Obese.

Step 2: the code.

```
#include <stdio.h>
int main(void) {
    double weight, height, bmi;
    printf("Weight and height: ");
    scanf("%lf %lf", &weight, &height);
    bmi = weight / (height * height);
    printf("BMI: %.1f\n", bmi);
    if (bmi < 18.5) {
        printf("Underweight\n");
    } else if (bmi < 25.0) {
        printf("Normal\n");
    } else if (bmi < 30.0) {
        printf("Overweight\n");
    } else {
        printf("Obese\n");
    }
```

```
    return 0;
}
```

Line by line.

- `double weight, height, bmi;` three boxes, all `double`, because 1.75 is not a whole number.
- `scanf("%lf %lf", &weight, &height);` two holes, two boxes, two `&`. `%lf` because they are `double`.
- `bmi = weight / (height * height);` the brackets make height × height happen first. Both are `double`, so it is real division; nothing is thrown away.
- `printf("BMI: %.1f\n", bmi);` one digit after the point. This line is not on the slide; it is there so you can see the number.
- The chain: the smallest limit first, because the conditions are "less than".

Step 3: run it.

You type	Screen shows
`70 1.75`	`BMI: 22.9` then `Normal`
`50 1.70`	`BMI: 17.3` then `Underweight`
`85 1.70`	`BMI: 29.4` then `Overweight`
`100 1.75`	`BMI: 32.7` then `Obese`

The question on the slide. Why is the second condition `bmi < 25.0` and not `bmi >= 18.5 && bmi < 25.0`? Because C only reaches the second condition if the first one was **false**. If `bmi < 18.5` is false, then `bmi` is already at least 18.5. The chain already knows that. Writing it again is not wrong, just unnecessary.

Why the order is smallest first. The conditions are "less than". `bmi < 18.5` catches the fewest values, so it goes first. If `bmi < 30.0` came first, every normal BMI would print `Overweight`. Same rule as slide 14, the other way round.

If your output is wrong, check:

1. `%lf` in `scanf`, with the L, and `&` on both boxes.
2. Brackets around `height * height`.
3. `\n` at the end of every `printf`.

Try it yourself. Change the program so it also prints `Weight: 70.0 kg` and `Height: 1.75 m` before the BMI. (Use `%.1f` and `%.2f`.)

Slide 19 — Worked example: electricity bill

A bill with **tiers**. The first 100 kWh cost 0.10 each. The next 100 cost 0.15 each. Everything above 200 costs 0.20 each. Most real bills and taxes work like this.

The code.

```
#include <stdio.h>
int main(void) {
    double kwh, bill;
    printf("kWh: ");
    scanf("%lf", &kwh);
    if (kwh <= 100) {
        bill = kwh * 0.10;
    } else if (kwh <= 200) {
        bill = 100 * 0.10 + (kwh - 100) * 0.15;
    } else {
```

```
        bill = 100 * 0.10 + 100 * 0.15 + (kwh - 200) * 0.20;
    }
    printf("Bill = %.2f\n", bill);
    return 0;
}
```

The three paths.

- Up to 100: every unit at 0.10.
- 101 to 200: the first 100 at 0.10, and **only the part above 100** at 0.15.
- Above 200: the first 100 at 0.10, the second 100 at 0.15, and **only the part above 200** at 0.20.

The mistake everyone makes: 150 kWh at 0.15 each = 22.50. Wrong. The first 100 units are still 0.10 each. Only the 50 above 100 are 0.15.

You type	Which path	The sum	Screen shows
`80`	first	80 × 0.10	`Bill = 8.00`
`100`	first	100 × 0.10	`Bill = 10.00`
`150`	second	100 × 0.10 + 50 × 0.15 = 10.00 + 7.50	`Bill = 17.50`
`200`	second	10.00 + 100 × 0.15	`Bill = 25.00`
`250`	third	10.00 + 15.00 + 50 × 0.20 = 10.00	`Bill = 35.00`

Here each path has its own **formula**, not just its own message. That is what conditionals are for: different calculations for different cases.

Try it yourself. Change the rule: the first 50 kWh are free, then 0.12 per kWh. Two paths. Test with `30` (expect `0.00`) and `150` (expect `12.00`: 100 units above 50, at 0.12).

Slide 20 — Code tracing exercise

Before you can write programs, you must be able to **read** them. On the quiz you cannot run the code. So practise tracing with a pen.

```
int x = 5;
int y = 12;
if (x * 2 > y) {
    printf("A");
} else if (y % x == 2) {
    printf("B");
} else {
    printf("C");
}
printf("D");
```

Trace it step by step.

1. `x` is 5, `y` is 12.
2. First condition: `x * 2 > y`. That is `10 > 12`. **False** (value 0). Skip the `A` block.
3. Second condition: `y % x == 2`. `12 % 5` is the remainder of 12 ÷ 5. 5 goes into 12 twice (10), remainder 2. So `2 == 2`. **True**. Print `B`. The chain stops; the `else` is not looked at.
4. `printf("D");` is **after** the chain, outside all the braces. It runs every time. Print `D`.

5. There is no `\n` anywhere, so both letters land on one line.

The four answers.

1. `x * 2 > y` is `10 > 12`, false, value `0`.
2. `y % x` is `12 % 5`, which is `2`.
3. The screen shows `BD`.
4. The `else if` branch runs.

The trap is the `printf("D")`. It looks like part of the chain, but look at the braces: the chain's last `}` closes before it. It belongs to nobody, so it always runs.

Try it yourself. Change `x` to 7 and trace again before running. (`x * 2 > y` is `14 > 12`, true. Screen shows `AD`.) Then change `x` to 6. (`12 > 12` is false; `12 % 6` is 0, not 2; so `else`. Screen shows `CD`.)

Slide 21 — In-class practice tasks

Five programs for the weekend. Each one is: ask, read, decide, print. Before you type, write in plain words what goes in, what comes out, and what the conditions are.

Task 1. The larger of two integers. Read two whole numbers, print the larger one.

Example run: you type `3 8`, the screen shows `Larger: 8`. You type `9 2`, it shows `Larger: 9`.

Hint: one `if (a > b)` with an `else`. (What happens when they are equal? Either answer is fine, and it prints that number.)

Task 2. Even or odd. Read a whole number, say whether it is even or odd.

Example run: you type `7`, the screen shows `7 is odd`. You type `10`, it shows `10 is even`.

Hint: `n % 2` is 0 for even numbers. Compare with `==`, two equals signs.

Task 3. A B C D F. Read a score, print the grade: 90 and above A, 80 and above B, 70 and above C, 60 and above D, otherwise F.

Example run: you type `83`, the screen shows `B`. You type `65`, it shows `D`. You type `12`, it shows `F`.

Hint: the chain from slide 13 with two more steps. Biggest number first.

Task 4. A tiny calculator with `switch`. Read an operation (`+`, `-` or `*`), then two numbers with decimals, print the result.

Example run: you type `+`, Enter, then `3 4`, Enter, the screen shows `Result: 7.00`. You type `*` then `2.5 4`, it shows `Result: 10.00`. You type `/` then `1 2`, it shows `Unknown operation`.

Hint: `char op;` and `scanf(" %c", &op);` with the space. Then `switch (op)` with `case '+':`, `case '-':`, `case '*':`, each with a `break`, and a `default`. Read the two numbers with `%lf`.

Task 5. Pay with overtime. Read hours worked and the hourly rate. Up to 40 hours are paid at the rate. Every hour above 40 is paid at one and a half times the rate.

Example run: you type `40 10`, the screen shows `Pay: 400.00`. You type `45 10`, it shows `Pay: 475.00` (40 × 10 = 400, plus 5 × 15 = 75). You type `30 12.5`, it shows `Pay: 375.00`.

Hint: `if (hours <= 40)` one formula, `else` another. In the `else`, the pay is 40 hours at the rate plus the hours above 40 at `rate * 1.5`. All boxes `double`.

Before you say a program is done: it compiles with `-Wall` and no warnings; every number printed has a label; you tested it with at least two inputs, one for each path; the code is indented.

Slide 22 — Quiz-style questions

Try each one on paper first. Then read the answer. A good answer gives the result **and** the rule that produced it.

Q1. `int a = 9, b = 4; printf("%d %d", a / b, a % b);` What is printed?

`2 1`. Both are `int`, so `9 / 4` is 2 with the .25 thrown away. `9 % 4` is the remainder: 4 goes into 9 twice (8), 1 is left.

Q2. `double avg = 7 / 2; printf("%.1f", avg);` Why is this not `3.5`?

It prints `3.0`. `7 / 2` is `int / int`, so it is calculated as 3 **before** it is stored in `avg`. Making the box a `double` does not help; the division has already happened. Fix: `7.0 / 2` or `(double) 7 / 2`, which gives `3.5`.

Q3.

```
if (x >= 0)
    printf("positive");
    printf("done");
```

Which line is always printed?

`done`. There are no braces, so only the **first** line after the `if` belongs to it. The second `printf` is indented to look like it belongs, but it does not. For `x = 5` the screen shows `positivedone`; for `x = -3` it shows `done`. With `-Wall` the compiler warns you: `this 'if' clause does not guard...`. Always write the braces.

Slide 23 — Week 2 summary

After this week you can:

- Choose a type: `int` for counts, `double` for decimals, `char` for one symbol.
- Explain why `7 / 2` is 3 and how `(double)` fixes it.
- Write a comparison and know it gives 1 or 0.
- Combine conditions with `&&`, `||`, `!`.
- Write `if`, `if/else`, an `else if` chain in the right order, a nested `if`, and a `switch` with `break` and `default`.
- Trace a program with a pen for one input and say exactly what it prints.

The three questions for every program. What data do I need? What type should store it? What decisions must the program make?

Before Tuesday.

1. **Do Knowledge check 4** on Blackboard, in the Week 2 module: this class's slides. Six questions, three attempts, best kept, worth 0%. Knowledge check 3 covers Tuesday's slides and is open too.
2. **Do the new practice tests** in the Practice folder: comparisons and logic, `if` and `else if`, `switch`, and whole programs for week 2. Unlimited attempts.
3. **Write the five programs** from slide 21. Bring them Tuesday, working or not.
4. **Quiz 1 comes back on Tuesday**, marked, and the mark appears in Blackboard.

Next week: loops. Doing something a hundred times without writing it a hundred times.